CYBERSECURITY CELLULE

Learning Roadmap & Activity Plan

November – February

Core sessions • Practical exercises • Interest-based advanced groups

Planning principle: begin with shared sessions and practical exercises. Do not launch or announce long-term group projects at the start. Observe participation and interests first; invite motivated students to form small groups later.

1. Programme overview

This four-month programme introduces first-year students to cybersecurity through short, practical sessions. The Cellule organisers are also developing their skills, so every session should be researched and tested in advance. The approach is collaborative: learn the concept, practise it safely, document what worked, and share the result.

Element	Plan
Duration	November to February; exact dates adapted to the university calendar.
Core sessions	12 sessions, normally one hour each, open to all interested first-year students.
Practice	Short guided exercises in sessions; small optional tasks between sessions.
Projects	No major group-project launch at the beginning. Consider projects only after observing interest and commitment.
Advanced track	Optional additional learning and project groups of 2–4 students, formed later if there is sufficient interest and supervision.
Final milestone	A friendly mini-CTF, project demonstrations if groups have launched, and programme feedback.

2. Guiding principles

- Keep the core programme accessible; do not split all participants into permanent beginner and advanced classes.
- Teach practical skills, not only definitions. Every core session should include an exercise.
- Prepare and test commands, tools, lab files, and expected results before each session.
- Do not announce advanced groups or require long-term projects at the start. First observe attendance, curiosity, and willingness to practise.
- Only form groups when students have expressed sustained interest and organisers can supervise them realistically.
- Use local labs, intentionally vulnerable training environments, or targets for which explicit permission has been granted.

3. Calendar at a glance

Month	Focus	Core sessions	Milestone
November	Foundations	1. Introduction to cybersecurity 2. Linux for cybersecurity 3. Networking fundamentals 4. Python for security	Low-pressure foundations challenge; observe engagement.
December	Security concepts	5. Security principles and common attacks 6. Cryptography essentials 7. Web security fundamentals	Short web-security case study; gather informal feedback.
January	Investigation and practice	8. Reconnaissance and network scanning 9. Defensive security and log analysis 10. Introduction to CTFs	Ask who wants to pursue additional work; explore interests and availability.
February	Consolidation and sharing	11. Guided practical challenge 12. Mini-CTF and programme review	If interest and time allow, start scoped advanced-group projects; otherwise plan continuation after February.

4. Detailed core-session roadmap

Session 1 — Introduction to cybersecurity

Suggested timing: November / Week 1

Learning objectives: Define cybersecurity and the CIA triad (confidentiality, integrity, availability); distinguish threats, vulnerabilities, and risks; introduce common roles and safe lab behaviour.

In-session exercise: Analyse short scenarios and identify the asset, threat, possible impact, and suitable protection.

Expected outcome: Students can explain the basic security goals and identify a simple risk.

Preparation: Prepare 5–6 short scenarios, a concise slide deck, and a glossary.

Session 2 — Linux for cybersecurity

Suggested timing: November / Week 2

Learning objectives: Practise navigation and file management; understand permissions, users, chmod/chown, processes, services, and logs.

In-session exercise: In a local Linux/VM environment, inspect permissions, change permissions on practice files, list processes, and locate a sample log.

Expected outcome: Students can explain a permission string and use a few essential commands safely.

Preparation: Test commands and prepare a worksheet with expected outputs. Avoid changing system-critical files.

Session 3 — Networking fundamentals

Suggested timing: November / Week 3

Learning objectives: Explain IP addresses, ports, TCP vs UDP, DNS, HTTP/HTTPS, and the basic path of a web request.

In-session exercise: Use ip/ipconfig, ping, and nslookup/dig where available; interpret a prepared DNS lookup or request flow.

Expected outcome: Students can describe what an IP address, port, DNS, and HTTPS do.

Preparation: Prepare a simple network diagram and OS-specific command alternatives.

Session 4 — Python for security

Suggested timing: November / Week 4

Learning objectives: Review variables, conditions, loops, functions, strings, lists, and reading text files.

In-session exercise: Create a small script that counts selected words/events in a prepared text or log file; alternatively check sample passwords against a stated policy without collecting real passwords.

Expected outcome: Students can modify a short script and explain its output.

Preparation: Provide starter code and a known input/output example.

Session 5 — Security principles and common attacks

Suggested timing: December / Week 1

Learning objectives: Discuss authentication vs authorisation, least privilege, phishing, malware, social engineering, and security controls.

In-session exercise: Review a fictional incident and map the likely weakness, impact, and prevention.

Expected outcome: Students can distinguish common attack types and propose reasonable mitigations.

Preparation: Use fictional examples; avoid asking students to submit personal credentials or sensitive information.

Session 6 — Cryptography essentials

Suggested timing: December / Week 2

Learning objectives: Explain hashing vs encryption, symmetric vs asymmetric cryptography, digital signatures, certificates, and HTTPS conceptually.

In-session exercise: Compare hashes of sample files; discuss why ordinary fast hashes alone are not appropriate for password storage.

Expected outcome: Students can choose whether a scenario calls for hashing, encryption, or a digital signature.

Preparation: Prepare harmless sample files and verify the hashing commands/tool in advance.

Session 7 — Web security fundamentals

Suggested timing: December / Week 3

Learning objectives: Introduce HTTP requests/responses, cookies, sessions, authentication, input validation, access control, and selected OWASP Top 10 concepts.

In-session exercise: Inspect a deliberately vulnerable training app or prepared case study and record possible weaknesses and fixes.

Expected outcome: Students can explain a basic web request and recognise common classes of weakness.

Preparation: Use only a local/authorised training app; prepare screenshots or a fallback case study.

Session 8 — Reconnaissance and network scanning

Suggested timing: January / Week 1

Learning objectives: Explain scope and authorisation, host/port discovery, service identification, and basic Nmap output.

In-session exercise: Scan a local lab or explicitly authorised training target and interpret a small result set.

Expected outcome: Students can explain what an open port suggests and why a scan result is not proof of a vulnerability.

Preparation: Provide a permitted target and exact scope; prepare sample output if lab access fails.

Session 9 — Defensive security and log analysis

Suggested timing: January / Week 2

Learning objectives: Introduce system logs, failed logins, suspicious events, monitoring, and a simple incident-response sequence.

In-session exercise: Analyse a provided log file, identify suspicious patterns, and write a short evidence-based finding.

Expected outcome: Students can cite evidence and distinguish observations from assumptions.

Preparation: Prepare synthetic logs with an answer key; do not use private real-world logs.

Session 10 — Introduction to CTFs

Suggested timing: January / Week 3

Learning objectives: Explain flags, challenge categories, hints, note-taking, and a systematic approach to problem-solving.

In-session exercise: Solve one beginner challenge together, such as basic encoding or file metadata analysis.

Expected outcome: Students know how to approach a challenge and document their reasoning.

Preparation: Test the challenge from a clean start and prepare progressive hints.

Session 11 — Guided practical challenge

Suggested timing: February / Week 1

Learning objectives: Combine concepts from Linux, networking, web security, logs, or basic scripting.

In-session exercise: Run a guided challenge in pairs; offer hints at set intervals and compare solution paths.

Expected outcome: Students apply more than one concept and explain their steps.

Preparation: Choose scope based on topics actually covered and available time.

Session 12 — Mini-CTF and programme review

Suggested timing: February / Week 2 or 3

Learning objectives: Consolidate learning, celebrate progress, and identify next learning goals.

In-session exercise: Run a friendly beginner mini-CTF; review solutions and invite brief demonstrations or reflections.

Expected outcome: Students can identify strengths, remaining gaps, and a next step.

Preparation: Prepare tested challenges at varied difficulty, clear rules, hints, and a feedback form.

5. Standard 60-minute session format

Time	Activity	Purpose
0–5 min	Quick recap	Recall the previous session and surface questions.
5–20 min	Concept introduction	Explain the core idea with one clear example.
20–45 min	Hands-on exercise	Let students practise individually or in pairs.
45–55 min	Challenge and discussion	Compare results, troubleshoot, and discuss safer approaches.
55–60 min	Wrap-up	Summarise key points and offer a small optional task.

6. Practice and project policy before advanced groups

During November and December, use short exercises that fit within a session or can be completed optionally. These are learning activities, not formal group projects. Do not ask students to commit to a multi-week project yet.

Period	What to do	What not to do
November	Teach foundations; use small individual/pair exercises; observe participation.	Do not form permanent advanced groups or promise projects.
December	Introduce security concepts and a short web-security case study; gather informal feedback.	Do not assume attendance alone means sustained interest.
January	Ask interested students about topics, time availability, and preferred commitment; use a small challenge to gauge readiness.	Do not create more groups than the organisers can support.
February	If enough students are interested, form 2–4-person groups and define a realistic first milestone.	Do not force a full project into the month if the schedule is too tight.

7. Optional advanced track — only after interest is confirmed

The advanced track is a later option, not a commitment at the start of the programme. Keep core sessions open to everyone. In January, invite interested students to express preferences. Form groups only when motivation, availability, and supervision are sufficient.

Possible group themes

Theme	Possible learning topics	Possible scoped project	Deliverable
A. Offensive security and web testing	Service enumeration; HTTP; sessions; access control; common web weaknesses; remediation.	Assess an intentionally vulnerable local web application and verify selected fixes.	Short technical report and safe lab demonstration.
B. Defensive security and incident response	Linux logs; failed logins; filtering; incident timelines; detection rules.	Write a small script to flag selected suspicious patterns in synthetic logs.	Script, test cases, sample alerts, and findings.
C. CTF, scripting and digital	CTF methodology; Python	Solve a curated set of	Challenge pack, documented

forensics	automation; hashes; metadata; file analysis.	intermediate challenges and create two beginner challenges with hints.	solutions, and demonstration.

Start with two groups if supervision is limited; add a third only if there are enough suitable participants and organisers can support them. Groups should normally contain 2–4 students and have one student coordinator.

Suggested advanced-group checkpoints

When	Activity	Output
November–December	No formal project track yet; students attend core sessions and explore topics.	Informal interests only.
January, after Session 10	Invite expressions of interest; discuss skills, preferred theme, availability, and commitment.	Provisional group list and topic preferences.
Late January / early February	Confirm group size, scope, safety boundaries, and achievable goal.	One-page project charter.
February	Run a first milestone or proof of concept if time allows; otherwise plan continuation.	Initial demonstration, progress note, and next steps.

If additional meetings are feasible, use one 60–90 minute meeting every two weeks. Do not add them until participant interest and organiser capacity are clear.

8. Project charter template for later use

Field	Questions to answer
Project title and theme	What will the group work on?
Problem / learning goal	What will the team learn or investigate?
Scope and safety	Which lab, sample dataset, or authorised environment is allowed? What is out of scope?
Team and roles	Who is in the group? Who coordinates? How are tasks shared?
Milestones	What can be completed in the first two weeks? What is the final target?
Deliverables	What will be shown: script, report, challenge pack, or demonstration?
Support needed	What tools, resources, or guidance are required?
Review date	When will the group show progress and decide the next step?

9. Organiser workflow and responsibilities

Before the session	During the session	After the session
Agree on 2–3 learning objectives; research the topic; test every command/exercise; prepare a solution and hints.	One organiser introduces concepts; the other supports the practical work. Alternate roles when useful.	Record attendance at a high level, questions, common errors, and improvements; save the exercise and solution in the club repository.

With two organisers, split preparation rather than creating a strict hierarchy: one can draft the concept explanation while the other builds the practical exercise, then both verify accuracy and test the whole session.

10. Repository structure suggestion

cod-club-cybersecurity/

```
├── README.md  — purpose, programme overview, participation and safety rules
├── roadmap/
│   └── november-february-roadmap.md
├── sessions/
│   ├── 01-introduction/
│   ├── 02-linux/
│   └── ...
├── exercises/
├── projects/  — add templates only when projects are confirmed
├── advanced-groups/  — create once groups are actually formed
└── resources/  — curated references and learning links
```

11. Success indicators by the end of February

- Participants can explain core cybersecurity concepts and complete basic Linux/networking exercises.
- Participants have practised Python or structured file/log analysis at an introductory level.
- Students can identify common web-security weaknesses and describe appropriate mitigations.
- Students have experienced a beginner CTF and know how to document their reasoning.
- The Cellule has reusable session materials, exercises, answer keys, and feedback for the next cohort.
- Any advanced groups that are formed have a clearly scoped task, documented progress, and a realistic continuation plan.

12. Final planning checklist

- ☐ Confirm the actual university dates and any exam/holiday weeks.
- ☐ Prepare and test Session 1 before the programme starts.
- ☐ Prepare Linux and networking labs with fallback screenshots or sample output.
- ☐ Create a simple attendance/feedback tracker.
- ☐ Keep November and December activities small and accessible.
- ☐ After Session 10, ask for expressions of interest in optional advanced work.
- ☐ Form only the number of advanced groups that can be supported.
- ☐ Review the programme at the end of February and decide what continues.